Simulator Edukasi Penjadwalan CPU dan Sinkronisasi Producer-Consumer

Laporan Tugas Data Structure

Process Scheduling
IPC & Classical Problems

Nama: Blessya
NIM: 001202400107
Kelas: Cyber Security
Institusi: President University
Dosen pengampu: Mam Ocha

2026

KATA PENGANTAR

Laporan ini membahas pembuatan simulator edukasi untuk materi Process Scheduling serta IPC & Classical Problems. Pembahasan difokuskan pada FCFS, Round Robin, dan Producer-Consumer agar aturan algoritma serta perubahan data dapat diamati melalui contoh yang terbatas dan mudah ditelusuri.

Isi laporan mencakup landasan teori, rancangan aplikasi, implementasi, dan pengujian. Data percobaan, kode sumber, serta petunjuk menjalankan program disertakan untuk mendukung pemeriksaan ulang. Pembatasan model dan lingkungan pengujian dinyatakan pada bagian analisis serta penutup.

Pengembangan kode, penyusunan dokumentasi, dan sebagian pemeriksaan otomatis dibantu asisten AI, termasuk Codex. Rujukan teori berasal dari sumber primer yang dicantumkan dalam daftar pustaka. Angka hasil diperiksa terhadap perhitungan manual, trace, dan keluaran pengujian yang disertakan. Penggunaan bantuan ini dinyatakan untuk menjelaskan proses penyusunan secara terbuka.

Blessya

ABSTRAK

Proyek ini menghasilkan aplikasi lokal untuk memperagakan dua materi Sistem Operasi, yaitu Process Scheduling dan IPC & Classical Problems. FCFS serta Round Robin digunakan untuk menunjukkan urutan giliran CPU dan perhitungan waktu tunggu, turnaround, serta respons. Producer-Consumer menggunakan thread Python, buffer terbatas, semaphore, dan mutex untuk menunjukkan koordinasi akses data bersama.

Metode evaluasi meliputi perhitungan manual, pengujian fungsi dan HTTP, pemeriksaan browser, serta analisis lima belas run buffer. Pada contoh tiga proses, FCFS dan Round Robin quantum dua menghasilkan rata-rata waktu tunggu 3,33 dan turnaround 6,33; respons rata-rata masing-masing 3,33 dan satu. Seluruh run buffer yang dicatat menyelesaikan target delapan barang dengan kapasitas terjaga dan tanpa thread tertinggal.

Parameter jeda pekerja dibedakan dari interval kejadian yang terukur. Pada dua puluh interval produksi awal skenario produsen cepat, masukan 10 ms menghasilkan interval 14,54-21,82 ms. Visualisasi buffer merupakan pemutaran ulang log setelah thread selesai. Hasil mendukung penggunaan aplikasi sebagai alat peraga pada kasus yang diuji, tanpa mengklaim simulasi seluruh OS, IPC dua proses terpisah, atau peningkatan hasil belajar yang belum diteliti.

Kata kunci: FCFS, Round Robin, penjadwalan CPU, Producer-Consumer, semaphore, mutex.

DAFTAR ISI

BAB I PENDAHULUAN 5
1.1 Latar belakang 5
1.2 Rumusan masalah 5
1.3 Tujuan dan ruang lingkup 6
BAB II LANDASAN TEORI 7
2.1 Proses dan perubahan keadaan 7
2.2 PCB dan context switch 8
2.3 Kriteria penjadwalan 9
2.4 First Come First Served 10
2.5 Round Robin 11
2.6 Definisi dan rumus metrik 12
2.7 Thread dan data bersama 13
2.8 Syarat critical section 14
2.9 Semaphore, mutex dan buffer terbatas 15
2.10 Operasi wait dan signal 16
BAB III PERANCANGAN SISTEM 18
3.1 Arsitektur aplikasi 18
3.2 Representasi proses dan waktu 19
3.3 Kontrak masukan dan validasi 19
3.4 Rancangan Producer-Consumer 20
3.5 Pencatatan kejadian dan konsistensi data 21
BAB IV IMPLEMENTASI DAN PENGUJIAN 23
4.1 Lingkungan dan prosedur pelaksanaan 23
4.2 Antarmuka penjadwalan CPU 23
4.3 Antarmuka Producer-Consumer 24
4.4 Strategi dan cakupan pengujian 25
4.5 Verifikasi manual FCFS 26
4.6 Verifikasi manual Round Robin 27
4.7 Pengaruh quantum 29
4.8 Pekerjaan panjang dan waktu idle 30
4.9 Percobaan produsen lebih cepat 31
4.10 Percobaan konsumen lebih cepat 32
4.11 Jeda sama dan variasi thread 33
4.12 Jeda masukan dan interval kejadian aktual 34
4.13 Pengujian dan perbaikan 35
BAB V PENUTUP 36
5.1 Kesimpulan 36
5.2 Keterbatasan 36
5.3 Saran pengembangan 36
DAFTAR PUSTAKA 37

BAB I PENDAHULUAN

1.1 Latar belakang

Belajar Sistem Operasi adalah alat peraga lokal untuk mengamati pembagian giliran CPU dan pertukaran barang melalui buffer terbatas. Program menyediakan dua materi inti: Process Scheduling melalui FCFS dan Round Robin, serta IPC & Classical Problems melalui Producer–Consumer. Pengguna dapat mengubah masukan, menjalankan percobaan, dan membaca urutan kejadian bersama penjelasan hasilnya.

Kebutuhan proyek berasal dari tugas yang meminta sebuah sistem sederhana untuk tujuan pendidikan dengan minimal dua materi Sistem Operasi. Laporan harus memuat penjelasan materi serta dokumentasi sistem dengan panjang minimal dua puluh halaman. Pemilihan dua materi ini memberikan dua bentuk pengamatan yang berbeda: scheduling menghasilkan urutan waktu yang dapat dihitung ulang, sedangkan Producer–Consumer memperlihatkan interaksi pekerja yang berbagi penyimpanan.

Masalah belajar yang ingin dibantu adalah kesulitan menghubungkan istilah dengan kejadian konkret. Angka waktu tunggu kurang bermakna jika pembaca tidak mengetahui kapan proses datang dan kapan mulai berjalan. Demikian pula, istilah semaphore lebih mudah dijelaskan setelah pengguna melihat produsen harus menunggu ketika izin slot kosong belum tersedia. Karena itu, tampilan berangkat dari contoh kecil dan menyediakan informasi rinci sebagai bagian yang dapat dibuka ketika dibutuhkan.

Sistem dibangun sebagai aplikasi Python dengan antarmuka browser. Seluruh fungsi inti aplikasi memakai pustaka standar Python serta HTML, CSS dan JavaScript tanpa framework. Pengguna tidak perlu membuat akun, menghubungkan layanan luar, atau memasang package aplikasi. Program berjalan pada localhost dan tetap dapat digunakan tanpa koneksi internet setelah folder proyek tersedia.

Hasil utama proyek adalah aplikasi dengan dua tab materi, perhitungan manual yang cocok dengan kasus uji, rekaman buffer dari thread nyata, dan bukti pengujian melalui algoritma, HTTP serta browser. Laporan ini menjelaskan pilihan model, implementasi, percobaan, temuan perbaikan, dan batas penarikan kesimpulan. Keberhasilan yang dilaporkan berarti sesuai pemeriksaan yang dilakukan, bukan pembuktian bahwa setiap kemungkinan input atau jadwal thread sudah diuji.

1.2 Rumusan masalah

Permasalahan yang dibahas adalah bagaimana memperagakan perubahan urutan eksekusi pada FCFS dan Round Robin, bagaimana menjaga buffer Producer-Consumer agar kapasitas dan urutan barang konsisten, serta bagaimana menyajikan hasil dengan penjelasan yang dapat ditelusuri ke kode dan data percobaan. Ketiga pertanyaan tersebut membatasi evaluasi pada perilaku sistem yang benar-benar diimplementasikan.

Penelitian proyek ini tidak menguji efektivitas pembelajaran pada sekelompok peserta dan tidak membandingkan performa kernel sistem operasi. Evaluasi dilakukan melalui perhitungan manual, pengujian program, pemeriksaan antarmuka, serta analisis trace. Oleh karena itu, istilah edukasi menyatakan tujuan penggunaan alat peraga, bukan hasil pengukuran peningkatan prestasi belajar.

1.3 Tujuan dan ruang lingkup

Tujuan pertama adalah menunjukkan bagaimana aturan pemberian giliran mengubah waktu mulai dan selesai proses. Tujuan kedua adalah menunjukkan bagaimana produsen dan konsumen dapat berbagi buffer tanpa melampaui kapasitas atau mengambil barang dari tempat kosong. Tujuan ketiga adalah mendokumentasikan rancangan, implementasi, dan pengujian sehingga hubungan antara teori dan perilaku program dapat diperiksa kembali.

Persyaratan	Pemenuhan dalam sistem
Minimal dua materi	Penjadwalan CPU dan Producer–Consumer
Tujuan pendidikan	Contoh kecil, Gantt chart, pemutaran log buffer
Penjelasan materi	Aturan, rumus, analogi, batas model
Dokumentasi	Arsitektur, input, screenshot dan pengujian
Laporan minimal 20 halaman	Jumlah halaman PDF diperiksa setelah render

Simulator scheduling menggunakan satu CPU atau core konseptual. Setiap proses memiliki waktu datang dan satu burst CPU yang diketahui. Tidak ada operasi I/O di tengah pekerjaan, tidak ada biaya waktu context switch, dan tidak ada perubahan prioritas pada dua algoritma inti. Waktu berupa bilangan bulat dalam satuan simulasi; angka tersebut tidak dimaksudkan sebagai waktu aktual CPU laptop.

Producer–Consumer utama menggunakan satu thread produsen dan satu thread konsumen dalam satu proses Python. Barang berupa label teks, bukan file atau pesan antarprogram. Kapasitas dan jumlah barang dibatasi agar percobaan singkat. Antarmuka memutar ulang hasil setelah thread selesai; kecepatan pemutaran dipisahkan dari waktu kejadian sebenarnya.

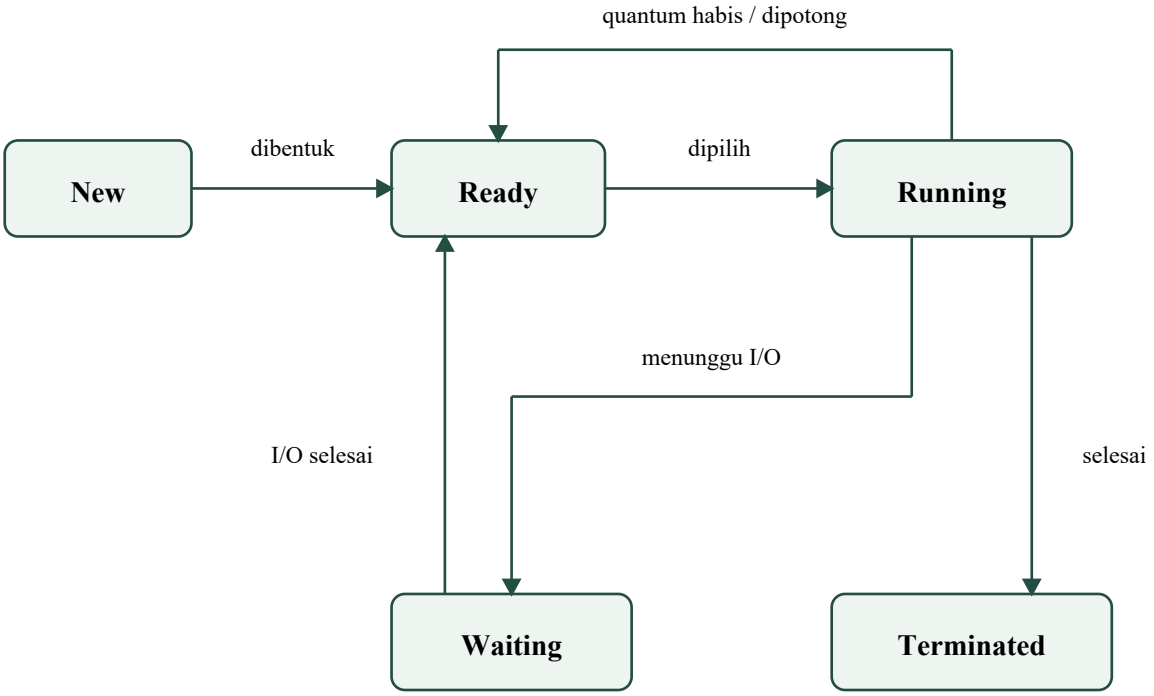
Kode tambahan dari versi sebelumnya, termasuk algoritma scheduling lain dan modul deadlock, dipertahankan sebagai materi eksplorasi. Tautan menuju tampilan tambahan tersedia pada bagian bawah halaman. Materi tersebut tidak dipakai untuk memperluas klaim utama laporan. Dengan pembatasan ini, pembaca dapat memahami dua materi inti sebelum menjelajahi fitur yang lebih luas.

BAB II LANDASAN TEORI

2.1 Proses dan perubahan keadaan

Program adalah kumpulan instruksi yang tersimpan, sedangkan proses adalah instans aktivitas eksekusinya. Model ini membantu menjelaskan mengapa satu program dapat dijalankan lebih dari sekali dengan keadaan kerja yang berbeda. CPU tidak hanya membutuhkan instruksi program, tetapi juga informasi tentang posisi eksekusi dan data yang sedang dikerjakan. Dalam teori penjadwalan, pekerjaan yang dapat memakai CPU dibedakan dari pekerjaan yang masih menunggu suatu kejadian [1].

Lima keadaan yang lazim dipakai dalam model proses adalah new, ready, running, waiting, dan terminated. New menyatakan proses sedang dibentuk; ready menyatakan siap memperoleh CPU; running berarti instruksi sedang dijalankan; waiting berarti menunggu suatu kejadian; terminated berarti eksekusi selesai. Pembedaan ready dan waiting penting: keduanya tidak sedang memakai CPU, tetapi hanya proses ready yang dapat segera dipilih scheduler [7].



Gambar 2.1 Model keadaan proses. Diagram disusun ulang untuk penjelasan konsep [7].

Pada diagram, pemilihan scheduler memindahkan proses dari ready ke running. Proses yang dipotong dapat kembali ke ready, sedangkan proses yang meminta I/O dapat beralih ke waiting. Setelah kejadian yang ditunggu selesai, proses kembali menjadi ready; proses tersebut belum tentu langsung memperoleh CPU. Diagram menjelaskan model umum, bukan rekaman transisi yang seluruhnya diimplementasikan oleh aplikasi.

Sebagai contoh dari data aplikasi, pada waktu satu P1 masih berjalan dan P2 sudah tiba. P2 berada pada keadaan siap menunggu giliran, bukan menunggu data dari perangkat I/O. Pada FCFS, P2 tetap siap sampai waktu lima. Pada Round Robin quantum dua, P1 kembali ke

antrean setelah waktu dua, lalu P2 mulai berjalan. Contoh ini menghubungkan istilah ready dengan angka waktu tunggu yang muncul pada hasil.

Model scheduling proyek tidak memasukkan transisi I/O ke waiting. Arrival menandai saat pekerjaan mulai siap, sedangkan completion menandai saat kebutuhan burst terpenuhi. Tidak ditampilkannya I/O merupakan pembatasan model agar perhitungan dapat diperiksa dengan contoh kecil. Keadaan menunggu semaphore pada modul Producer-Consumer adalah peristiwa thread nyata yang berbeda dari antrean siap pada simulator CPU.

Kejadian contoh	Interpretasi yang benar
P2 sudah datang tetapi P1 masih berjalan	P2 siap dan menunggu giliran CPU
CPU belum memiliki proses yang datang	Interval IDLE pada model
Konsumen belum memperoleh izin full	Thread menunggu izin barang
Seluruh burst P3 telah dikerjakan	P3 selesai dalam model scheduling

Karena kedua modul memakai jenis waktu yang berbeda, angka arrival satu tidak berarti thread menunggu satu milidetik. Scheduling menggunakan jam abstrak; modul buffer merekam waktu eksekusi dengan perf_counter(). Pembedaan ini dipertahankan dalam tabel, grafik dan analisis agar kesimpulan dari kedua modul tidak dicampur menjadi ukuran kinerja komputer.

2.2 PCB dan context switch

Process Control Block atau PCB adalah struktur kernel yang menyimpan informasi terkait proses. Informasi tersebut antara lain keadaan, program counter, register CPU, informasi penjadwalan, pengelolaan memori, pencatatan penggunaan sumber daya dan status I/O. Ketika CPU berpindah proses, konteks yang diperlukan disimpan dan konteks proses berikutnya dipulihkan. Pergantian ini memerlukan pekerjaan sistem, sehingga waktunya merupakan overhead terhadap pekerjaan aplikasi [7].

Informasi konseptual	Mengapa diperlukan	Representasi pada simulator
Program counter dan register	Melanjutkan instruksi pada posisi yang tepat	Tidak diemulasikan
Keadaan dan antrean	Mengetahui proses yang dapat dipilih	Antrean dan sisa burst
Informasi memori dan I/O	Mengelola ruang alamat serta perangkat	Di luar model

Informasi konseptual	Mengapa diperlukan	Representasi pada simulator
Identitas proses	Menghubungkan aktivitas dengan pekerjaan	pid/nama proses
Penggunaan CPU	Mencatat pekerjaan yang telah dijalankan	Timeline dan metrik

Objek Process pada proyek bukan PCB kernel yang lengkap. Objek tersebut hanya menyimpan masukan yang diperlukan oleh algoritma: nama, waktu datang, burst, serta prioritas untuk modul tambahan. Round Robin juga menyimpan sisa burst dalam struktur data algoritma. Pemilihan informasi yang terbatas ini cukup untuk menghitung giliran tanpa meniru register mesin, tabel halaman, atau seluruh manajemen proses sistem operasi.

Pada contoh P1 yang terpotong setelah dua satuan, simulator cukup mengingat bahwa sisa pekerjaannya tiga. Saat dipilih kembali, sisa itulah yang dikurangi. Sistem operasi nyata harus memulihkan konteks instruksi dan data yang lebih lengkap. Dengan demikian, penyimpanan sisa burst memperagakan kebutuhan mengingat kemajuan pekerjaan, tetapi tidak boleh disebut sebagai implementasi context switch kernel.

Definisi operasional pergantian pada program dibatasi menjadi dua segmen berurutan dengan identitas proses berbeda, tanpa IDLE. Sebagai akibatnya, satu proses dengan burst lima dan quantum dua memiliki tiga giliran, tetapi nol pergantian antarproses. Ini sengaja dibedakan dari peristiwa timer habis dan dispatch pertama. Definisi yang sama dipakai untuk membandingkan FCFS dan Round Robin.

Jika sebuah analisis tambahan memberikan biaya hipotetis 0,1 satuan per pergantian, lima pergantian berarti 0,5 satuan biaya tambahan menurut definisi tersebut. Angka ini hanya ilustrasi perhitungan, bukan hasil program atau pengukuran laptop. Simulator yang diserahkan tidak menambahkan biaya itu, sehingga total waktu pada contoh tetap sembilan untuk kedua algoritma. Hasil model tidak digunakan untuk menaksir durasi context switch Windows.

2.3 Kriteria penjadwalan

Kriteria penjadwalan perlu dipilih sebelum membandingkan algoritma. Utilisasi menyatakan proporsi waktu CPU aktif; throughput menyatakan jumlah pekerjaan yang selesai per satuan waktu. Turnaround mengukur rentang dari kedatangan sampai selesai, waiting mengukur waktu dalam antrean siap, dan response berfokus pada respons pertama. Pembagian kriteria ini mengikuti pembahasan CPU scheduling pada sumber [8].

Kriteria	Arah yang biasanya diinginkan	Batas pembacaan hasil proyek
Utilisasi	Lebih tinggi	Rasio waktu aktif terhadap sumbu simulasi
Throughput	Lebih banyak selesai per satuan waktu	Tidak diukur sebagai performa laptop

Kriteria	Arah yang biasanya diinginkan	Batas pembacaan hasil proyek
Turnaround	Lebih pendek	Termasuk seluruh giliran proses
Waiting	Lebih pendek	Rumus sederhana berlaku pada model tanpa I/O
Response	Lebih pendek	Waktu sampai pertama memperoleh CPU

Perbedaan tujuan dapat menghasilkan pilihan yang berbeda. Pengguna aplikasi interaktif biasanya merasakan pentingnya respons awal, sedangkan suatu pekerjaan batch mungkin lebih menekankan waktu selesai. Laporan ini tidak memberi satu skor gabungan yang menyembunyikan perbedaan tersebut. Tiga rata-rata utama ditampilkan berdampingan agar perubahan ukuran dapat diperiksa secara terpisah.

Pada data P1, P2, dan P3, FCFS dan RR quantum dua sama-sama menyelesaikan tiga pekerjaan dalam sembilan satuan tanpa waktu kosong. Rasio throughput model untuk keduanya adalah 3/9 pekerjaan per satuan waktu. Namun, respons rata-ratanya berbeda: 3,33 dan satu. Contoh ini memperlihatkan bahwa kesamaan throughput atau total waktu tidak menjamin pengalaman menunggu awal yang sama.

Rata-rata juga perlu disertai hasil setiap proses. Pada contoh tersebut, waktu tunggu P1 meningkat dari nol menjadi empat ketika menggunakan RR, sedangkan P3 turun dari enam menjadi dua. Rata-rata tunggu keseluruhan tetap 3,33. Melihat rata-rata saja akan menyembunyikan distribusi perubahan tersebut. Oleh sebab itu, tabel hasil per proses merupakan bagian penting dari dokumentasi percobaan.

Quantum yang kecil memberi kesempatan bergiliran lebih sering, tetapi dapat meningkatkan frekuensi perpindahan. Quantum yang cukup besar dapat membuat setiap burst contoh selesai pada giliran pertama sehingga hasilnya menyerupai FCFS [8]. Hubungan ini digunakan sebagai hipotesis yang diperiksa pada data aplikasi. Pemilihan quantum optimal untuk sistem nyata berada di luar penelitian ini karena biaya perpindahan, I/O dan beban dinamis tidak dimodelkan.

2.4 First Come First Served

First Come First Served memberi giliran kepada pekerjaan berdasarkan kedatangannya. Pekerjaan yang sudah mendapat CPU berjalan sampai selesai pada model nonpreemptive ini. Sifat sederhana tersebut memudahkan penelusuran, tetapi pekerjaan panjang yang berada di depan dapat membuat pekerjaan pendek menunggu. Pembahasan kebijakan dasar penjadwalan dan pengukuran turnaround dapat ditemukan pada OSTEP bab Scheduling Introduction [2].

Implementasi fcfs() mengurutkan daftar proses dengan kunci pasangan arrival dan pid. Waktu datang merupakan kunci utama. Nama proses menjadi pemutus urutan ketika dua waktu datang sama. Aturan tambahan ini merupakan keputusan implementasi yang harus diketahui

saat membandingkan hasil dengan perhitungan manual. Nama dibandingkan sebagai teks, sehingga P10 dapat berada sebelum P2.

```
urutkan proses berdasarkan (arrival, nama)
jam = 0
untuk setiap proses:
    jika jam < arrival: catat IDLE sampai arrival
    mulai = maksimum(jam, arrival)
    selesai = mulai + burst
    catat segmen proses
    jam = selesai
```

Pseudocode tersebut menjelaskan aturan program sendiri. Setiap proses menghasilkan satu segmen aktif. Jika ada jeda kedatangan, program menambahkan segmen IDLE sebelum proses berikutnya. Dengan demikian, grafik memuat waktu kosong sekaligus waktu kerja. Fungsi pembentuk metrik kemudian membaca timeline yang sama; frontend tidak menghitung ulang algoritma secara terpisah.

Dalam demo, pengguna dapat mengubah lama P1 dari lima menjadi dua puluh empat untuk melihat dampak pekerjaan panjang. Nilai waktu tunggu proses berikutnya bertambah karena mereka belum memperoleh giliran. Fenomena yang diperlihatkan bukan kesalahan FCFS, melainkan akibat dari aturan tidak memotong pekerjaan aktif. Penjelasan ini penting agar pengguna membedakan perilaku algoritma dari bug implementasi.

FCFS berguna sebagai acuan karena urutannya mudah diverifikasi dengan penjumlahan sederhana. Acuan ini dipakai pada pengujian manual sebelum membandingkannya dengan Round Robin. Aplikasi tidak menyatakan FCFS selalu buruk atau selalu terbaik; penilaian bergantung pada data dan ukuran yang ingin diperhatikan.

2.5 Round Robin

Round Robin membagi CPU dalam giliran dengan batas waktu yang disebut quantum. Proses yang belum selesai setelah satu giliran dimasukkan lagi ke belakang antrean. Dalam literatur penjadwalan, pembagian waktu ini berkaitan dengan tujuan respons yang lebih cepat serta biaya pergantian pekerjaan [2]. Simulator memperlihatkan pembagian giliran, tetapi tidak menambahkan durasi biaya perpindahan pada jam model.

Fungsi round_robin() menyimpan sisa burst untuk setiap nama proses dan sebuah antrean siap. Lama satu giliran adalah nilai terkecil antara quantum dan sisa burst. Proses yang hanya memiliki satu satuan pekerjaan tersisa akan selesai dalam satu satuan walaupun quantum bernilai dua. Karena itu, tidak semua batang dalam Gantt chart harus memiliki lebar yang sama.

```
masukkan proses yang sudah datang ke antrean
selama masih ada pekerjaan:
    bila antrean kosong: lompat ke kedatangan berikutnya
    ambil proses dari depan antrean
    durasi = minimum(quantum, sisa burst)
    jalankan dan kurangi sisa burst
    masukkan proses yang tiba sampai akhir giliran
    jika proses belum selesai: antrekan kembali
```

Kedatangan tepat di batas quantum diterima sebelum proses aktif dimasukkan ulang. Sebagai contoh, jika P1 berjalan dari nol sampai dua dan P3 datang pada dua, P3 masuk bersama proses baru sebelum P1 kembali ke belakang. Aturan ini menentukan urutan contoh manual dan dilindungi oleh kasus uji khusus. Mengubahnya tanpa memperbarui penjelasan akan membuat hasil sulit dibandingkan.

Batas setiap giliran tetap disimpan pada timeline walaupun hanya satu proses yang tersedia. Satu proses dengan burst lima dan quantum dua menghasilkan tiga giliran: dua, dua, dan satu. Namun, pergantian antarprosesnya nol karena identitas proses tidak berubah. Perbedaan antara habisnya quantum dan pergantian proses merupakan salah satu perbaikan penting proyek ini.

Pengguna dapat mencoba quantum satu, dua, tiga, lalu lima pada contoh awal. Hasil percobaan memperlihatkan bahwa respons, waktu tunggu, dan jumlah pergantian tidak harus berubah dengan pola yang sama. Kesimpulan diambil dari tabel hasil per input, bukan dari anggapan bahwa quantum terkecil selalu paling baik.

2.6 Definisi dan rumus metrik

Metrik menjawab pertanyaan berbeda. Waktu selesai menjelaskan kapan pekerjaan berakhir. Turnaround mencakup seluruh rentang sejak kedatangan. Waktu tunggu hanya menghitung bagian rentang tersebut ketika proses belum memakai CPU dalam model tanpa I/O. Respons berfokus pada giliran pertama, sehingga berbeda dari waktu selesai untuk proses yang terpotong beberapa kali.

Ukuran	Rumus dalam program
Completion	Akhir segmen terakhir proses
Turnaround	Completion dikurangi arrival
Waiting	Turnaround dikurangi burst
Response	Awal segmen pertama dikurangi arrival
Rata-rata	Jumlah nilai semua proses dibagi jumlah proses

Contoh P2 pada FCFS: waktu datang satu, burst tiga, mulai lima, dan selesai delapan. Turnaround adalah tujuh. Dari tujuh satuan itu, tiga dipakai untuk bekerja sehingga empat

sisanya adalah waktu tunggu. Karena hanya ada satu giliran, waktu tunggu dan respons sama-sama empat. Persamaan angka tersebut tidak berlaku umum untuk Round Robin.

Pada Round Robin quantum dua, P2 mulai pada dua dan selesai pada delapan. Responsnya satu, tetapi total waktu tunggunya empat. P2 sempat bekerja, kembali mengantre, lalu bekerja lagi. Contoh ini menunjukkan mengapa satu angka saja tidak cukup untuk menggambarkan pengalaman suatu proses. Program menampilkan tiga rata-rata utama dan menyediakan detail per proses.

Fungsi build_metrics() memperoleh waktu pertama dan terakhir dari timeline. Hasil numerik tidak berasal dari posisi visual batang atau lebar piksel. Rata-rata dibulatkan dua angka desimal untuk tampilan, sementara waktu tiap proses tetap berupa integer. Perbandingan manual menggunakan pecahan terlebih dahulu kemudian membulatkannya, sehingga tidak menumpuk kesalahan pembulatan dari langkah sebelumnya.

Context switch pada laporan didefinisikan sebagai peralihan langsung antara dua proses yang berbeda. Dispatch pertama dan peralihan ke atau dari IDLE tidak dihitung. Definisi ini dijelaskan karena istilah yang sama dapat diberi batas pengukuran berbeda. Angka tersebut bukan pengukuran context switch sistem operasi laptop, dan tidak boleh dipakai untuk menghitung overhead nyata tanpa model tambahan.

2.7 Thread dan data bersama

Thread adalah alur eksekusi di dalam proses. Beberapa thread dalam proses yang sama dapat mengakses ruang data bersama. Hal ini memberi kesempatan bekerja secara konkuren, sekaligus menuntut koordinasi ketika beberapa thread mengakses data yang sama [3]. Pada proyek ini, data bersama tersebut adalah daftar barang dalam buffer.

Tanenbaum membedakan pengelompokan sumber daya pada proses dari alur eksekusi thread. Thread dalam satu proses berbagi ruang alamat dan sumber daya seperti berkas terbuka, tetapi mempunyai keadaan eksekusi seperti program counter, register, dan stack masing-masing [10]. Pada aplikasi ini, daftar buffer merupakan data bersama, sedangkan pengulangan produksi dan konsumsi dijalankan oleh alur pekerja yang berbeda. Sampel buku tersebut dipakai untuk konsep dasar thread, tanpa menganggapnya sebagai bukti buku pegangan mata kuliah ini.

Modul threading Python menyediakan Thread, Lock, Semaphore, dan Event. Thread menjalankan fungsi pekerja, Lock mengatur akses eksklusif, Semaphore mengatur izin yang dihitung, sedangkan Event digunakan untuk memberi sinyal pembatalan. Dokumentasi Python juga membedakan konkurensi thread dari kemampuan eksekusi paralel CPU pada build CPython yang menggunakan GIL [4]. Demo ini tidak digunakan sebagai benchmark peningkatan kecepatan komputasi.

Produsen menjalankan loop untuk membuat label barang, menunggu jeda produksi, kemudian meminta buffer menerima barang tersebut. Konsumen mempunyai kuota pengambilan dan jeda konsumsi. Satu item memiliki identitas seperti item-1.3, yang menyatakan barang ketiga

dari produsen pertama. Label ini membuat urutan barang dapat dibandingkan antara rekaman produksi dan konsumsi.

Bagian	Data atau tanggung jawab
Produsen	Membuat label dan meminta put
Konsumen	Meminta get sesuai kuota
BoundedBuffer	Menjaga isi, kapasitas dan semaphore
Recorder	Menyimpan kejadian dan snapshot
Pengendali percobaan	Memulai, menunggu dan membatalkan pekerja

Antarmuka utama menetapkan satu produsen dan satu konsumen agar hubungan sebab akibat mudah dilihat. Backend tetap mendukung beberapa pekerja untuk mempertahankan kemampuan tambahan dan menguji konsistensi pencatatan. Pengujian dengan banyak pekerja tidak mengubah materi utama; tujuannya membantu menemukan kesalahan yang mungkin tersembunyi pada urutan eksekusi sederhana.

Sebutan IPC & Classical Problems pada tugas dipenuhi melalui masalah klasik Producer–Consumer dan mekanisme sinkronisasinya. Implementasinya tidak membuat dua proses OS yang terpisah, tidak menggunakan pipe antarprogram, dan tidak melakukan komunikasi jaringan antarpekerja. HTTP hanya menghubungkan antarmuka browser dengan server aplikasi. Batas ini dinyatakan agar konsep demonstrasi tidak salah dijelaskan saat presentasi.

2.8 Syarat critical section

Critical section adalah bagian program yang mengakses keadaan bersama dan perlu dikoordinasikan. Tiga syarat klasik solusi masalah critical section ialah mutual exclusion, progress, dan bounded waiting. Mutual exclusion melarang dua eksekusi berada pada bagian kritis yang sama secara bersamaan. Progress menuntut pemilihan peserta berikutnya tidak ditunda tanpa batas ketika bagian kritis kosong dan ada peserta yang ingin masuk. Bounded waiting membatasi berapa kali peserta lain boleh mendahului setelah permintaan masuk dibuat [9].

Syarat	Pertanyaan pemeriksaan	Hubungan dengan buffer
Mutual exclusion	Adakah dua pihak mengubah data bersama sekaligus?	Mutex mengelilingi daftar barang dan snapshot
Progress	Dapatkah pekerjaan yang memenuhi syarat melanjutkan?	Penantian izin berlangsung di luar mutex
Bounded waiting	Apakah jumlah pendahuluan dibatasi?	Tidak dibuktikan secara formal oleh pengujian ini

Contoh kesalahan yang harus dicegah adalah pola periksa-lalu-ubah tanpa perlindungan. Dua produsen dapat sama-sama membaca bahwa masih ada satu slot, kemudian keduanya menambahkan barang. Keputusan berdasarkan hasil pemeriksaan lama tidak lagi sesuai keadaan terbaru. Karena pemeriksaan dan tindakan dapat disela oleh pekerja lain, penggunaan variabel penghitung biasa saja tidak cukup untuk menyatakan izin secara aman.

Pada implementasi ini, izin empty diperoleh lebih dahulu, kemudian penambahan item, pembaruan penghitung dan penyalinan snapshot dilakukan di dalam mutex. Pengamatan yang direkam menjadi satu rangkaian perubahan yang terlindungi. Konsumen melakukan urutan sepadan dengan full. Pengujian merekonstruksi daftar barang setelah setiap event untuk memastikan bahwa log dan keadaan yang diklaim saling sesuai.

Tidak menunggu empty atau full sambil memegang mutex mencegah pola saling menghalangi yang langsung timbul apabila pemegang mutex menunggu slot yang hanya bisa dibebaskan pasangannya. Walaupun demikian, pernyataan ini tidak membuktikan semua jadwal thread pasti maju dalam waktu tertentu. Scheduler, kegagalan pekerja, dan sifat primitif sinkronisasi tetap perlu dipertimbangkan ketika membahas liveness.

Dokumentasi Python tidak menjanjikan urutan FIFO bagi thread yang dibangunkan oleh Semaphore. Karena itu, aplikasi tidak mengklaim pembuktian bounded waiting atau ketiadaan starvation untuk semua jumlah pekerja [4]. Tampilan inti memakai satu produsen dan satu konsumen, sedangkan tes beberapa pekerja menjadi bukti empiris tambahan. Timeout dan pembatalan adalah penanganan kegagalan operasional, bukan pengganti bukti fairness.

Pemisahan safety dan liveness membantu membaca hasil. Kapasitas tidak terlampaui, barang tidak hilang, dan urutan FIFO terjaga merupakan sifat safety yang diuji dari trace. Seluruh pekerja selesai pada run yang dicatat merupakan pengamatan tentang penyelesaian run tersebut. Kesimpulan tidak diperluas menjadi jaminan bahwa seluruh lingkungan dan interleaving akan selalu selesai tanpa kendala.

2.9 Semaphore, mutex dan buffer terbatas

Semaphore menyimpan jumlah izin yang tersedia. Operasi memperoleh izin dapat membuat pemanggil menunggu ketika izin belum tersedia. Operasi pelepasan mengembalikan izin. Pada masalah buffer terbatas, satu semaphore dapat mewakili slot kosong dan satu lagi barang yang siap diambil. Pola Producer–Consumer dengan semaphore dibahas pada OSTEP bab Semaphores [5].

Mutex mempunyai tugas berbeda: melindungi critical section agar hanya satu thread mengubah data buffer pada satu waktu. Penggunaan lock di sekitar operasi terhadap data bersama membantu membuat rangkaian perubahan tersebut terlihat sebagai satu tindakan yang utuh terhadap thread lain [6]. Dalam proyek ini, perubahan isi dan perekaman snapshot berada dalam perlindungan yang sama.

Mekanisme	Nilai awal	Peran pada aplikasi
empty	Kapasitas buffer	Izin menaruh satu barang
full	Nol	Izin mengambil satu barang
mutex	Tidak terkunci	Melindungi isi dan snapshot
cancel	Belum diset	Menghentikan pekerja jika gagal

Urutan operasi produsen adalah memperoleh izin empty, masuk ke mutex, menambahkan barang dan log, keluar dari mutex, lalu melepas izin full. Konsumen memperoleh full, masuk ke mutex, mengambil barang terdepan dan mencatat log, lalu melepas empty. Thread tidak menunggu izin empty atau full sambil memegang mutex. Pemisahan urutan ini memungkinkan pasangan pekerja tetap mengubah buffer saat pekerja pertama sedang menunggu.

Invariant fisik buffer adalah jumlah barang tidak kurang dari nol dan tidak lebih besar dari kapasitas. Nilai semaphore tidak ditampilkan sebagai salinan langsung jumlah barang karena izin dapat sedang dipegang oleh thread di antara tahap memperoleh izin dan mengubah buffer. Visualisasi menggunakan snapshot daftar barang yang diambil di dalam mutex, sehingga yang dilihat adalah keadaan buffer yang dicatat, bukan perkiraan dari semaphore.

Keadaan menunggu tidak otomatis berarti deadlock. Produsen yang menunggu slot dapat dilanjutkan setelah konsumen mengambil barang. Sebaliknya, konsumen dapat dilanjutkan setelah produsen memasukkan barang. Pemeriksaan akhir menuntut semua barang selesai dipindahkan dan seluruh thread berhenti. Jika batas waktu terlampaui, hasil ditandai gagal dan pekerja diberi sinyal pembatalan.

2.10 Operasi wait dan signal

Dalam naskah EWD123, Dijkstra menggunakan operasi P dan V pada semaphore. P mengurangi izin ketika pengurangan dapat dilakukan tanpa menghasilkan nilai negatif; jika belum dapat, penyelesaiannya menunggu. V menambah izin. Keputusan memperoleh izin dan perubahan nilainya harus dipandang sebagai operasi tak terpisahkan, sehingga dua pemanggil tidak memakai izin yang sama [11].

Pseudocode berikut memakai nama wait dan signal untuk memudahkan hubungan dengan acquire dan release di Python. Bentuk ini adalah spesifikasi perilaku abstrak, bukan kode Python atau implementasi kernel. Bagian atomik beserta mekanisme tidur dan membangunkan membutuhkan dukungan primitif sinkronisasi; pseudocode tidak boleh diterjemahkan menjadi pemeriksaan variabel biasa tanpa perlindungan.

```
wait(S):
    secara atomik:
        jika izin S tersedia: ambil satu izin dan kembali
        jika tidak: daftarkan pemanggil untuk menunggu
    pemanggil tidur sampai memperoleh izin

signal(S):
    secara atomik:
        kembalikan satu izin
        izinkan satu penunggu yang memenuhi syarat melanjutkan
```

Untuk buffer berkapasitas N, jumlah izin empty dimulai dari N dan full dari nol. Produsen memerlukan empty sebelum menaruh barang, lalu memberi full setelah barang sudah tersimpan. Konsumen memerlukan full sebelum mengambil barang, lalu memberi empty setelah slot sudah bebas. Mutex tetap diperlukan karena izin kapasitas tidak membuat rangkaian perubahan struktur data menjadi satu operasi atomik.

```
produsen:                konsumen:
    buat barang              wait(full)
    wait(empty)              lock(mutex)
    lock(mutex)              ambil barang terdepan
    masukkan barang          catat snapshot
    catat snapshot           unlock(mutex)
    unlock(mutex)            signal(empty)
    signal(full)             olah barang
```

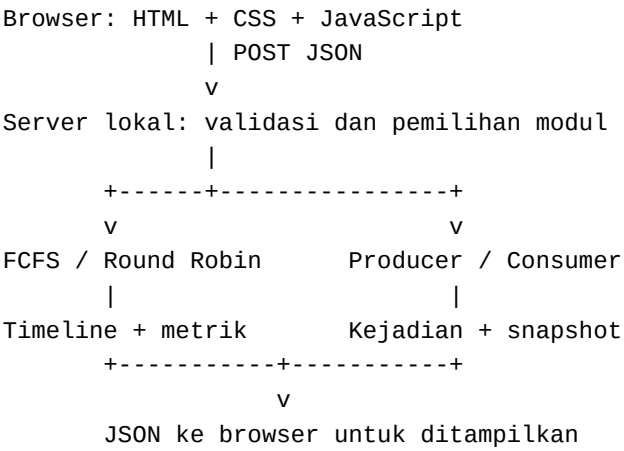
Contoh kapasitas dua dimulai dengan buffer kosong. Setelah satu barang masuk, terdapat satu barang yang dapat diambil. Setelah dua barang masuk, penambahan ketiga harus menunggu konsumen. Namun, pada saat sebuah pekerja sudah memperoleh izin tetapi belum masuk mutex, nilai semaphore tidak harus sama dengan jumlah item fisik. Karena alasan itu, aplikasi menampilkan snapshot daftar barang, bukan membaca nilai internal semaphore sebagai ukuran isi.

Ketika buffer kosong, konsumen yang belum mendapat full menunggu perubahan yang dapat dilakukan produsen. Ketika penuh, produsen menunggu konsumen menyediakan slot. Keadaan ini adalah sinkronisasi normal selama pasangan pekerja dapat melanjutkan. Urutan yang keliru, misalnya mengunci mutex lalu menunggu empty pada buffer penuh, dapat mencegah konsumen melakukan pengambilan yang diperlukan. Perbedaan tersebut menjadi alasan penempatan operasi pada pseudocode.

BAB III PERANCANGAN SISTEM

3.1 Arsitektur aplikasi

Arsitektur dipisahkan menjadi tiga bagian: antarmuka, endpoint HTTP, dan modul algoritma. Pemisahan ini menjaga perhitungan tetap berada pada satu sumber. Browser menangani masukan dan tampilan, server memvalidasi permintaan, dan modul Python menghasilkan data simulasi. Tidak ada database karena seluruh percobaan dapat dijalankan dari parameter yang dimasukkan pengguna.



Endpoint /api/scheduling menerima daftar proses dan nama algoritma. Untuk perbandingan dua algoritma inti, antarmuka mengirim dua permintaan dengan data proses yang sama. Respons masing-masing berisi hasil tunggal. Pendekatan ini menggunakan kemampuan backend yang sudah ada tanpa memaksa halaman utama menampilkan seluruh algoritma tambahan.

Endpoint /api/ipc/producer-consumer menjalankan percobaan sampai selesai atau sampai gagal. Respons memuat parameter, jumlah produksi dan konsumsi, isi tertinggi, kejadian menunggu, status thread, dan daftar kejadian. Setelah respons diterima, browser menyediakan kontrol pemutaran. Oleh karena itu, kelambatan pemutaran visual tidak memengaruhi penjadwalan thread pada percobaan yang sudah selesai.

Server memakai ThreadingHTTPServer agar permintaan dapat ditangani oleh thread server. Thread server berbeda dari thread produsen dan konsumen yang dipakai dalam percobaan. Semua objek buffer dibuat untuk masing-masing pemanggilan run_demo(), sehingga dua percobaan tidak berbagi daftar barang yang sama. Data ekspor JSON diunduh ke pengguna melalui browser; aplikasi tidak mengirimkannya ke layanan luar.

Berkas web disajikan dari direktori web. Dokumen laporan dan bukti pengujian berada di direktori terpisah dalam proyek. Dokumen tersebut dibuka melalui pengelola berkas. Pembatasan ini menjaga aplikasi tetap kecil serta membuat lokasi kode, dokumentasi, dan hasil percobaan mudah dikenali.

3.2 Representasi proses dan waktu

Proses dapat dipahami sebagai program yang sedang dieksekusi. Sistem operasi membagi penggunaan CPU di antara pekerjaan yang berjalan sehingga beberapa program dapat mengalami kemajuan dalam periode yang sama. Pengertian proses dan pembagian waktu ini menjadi landasan untuk memisahkan program yang tersimpan dari aktivitas eksekusinya [1].

Pada aplikasi, proses direpresentasikan oleh objek Process. Objek ini menyimpan nama, arrival, burst, dan field prioritas untuk kompatibilitas dengan materi tambahan. FCFS dan Round Robin tidak menggunakan field prioritas. Nama dipakai untuk menghubungkan segmen pada grafik dengan baris hasil perhitungan. Karena itu, dua proses tidak boleh memiliki nama sama dalam satu percobaan.

Istilah	Makna pada model	Contoh
Nama proses	Identitas pekerjaan	P1
Arrival	Waktu pekerjaan siap	0
Burst	Kebutuhan waktu CPU	5
Segmen	Interval penggunaan CPU	P1 dari 0 sampai 5
IDLE	Tidak ada proses siap	CPU kosong 0 sampai 5

Jam simulasi dimulai dari nol. Jika proses pertama baru datang pada waktu lima, interval nol sampai lima dicatat sebagai IDLE. Nama IDLE dikhususkan bagi keadaan ini dan ditolak sebagai nama proses pengguna. Pembatasan tersebut diperlukan karena penghitungan metrik dan pewarnaan grafik membedakan segmen proses dari segmen kosong berdasarkan penanda itu.

Interval ditulis dengan awal termasuk dan akhir tidak termasuk, misalnya [0,5). Durasi interval adalah lima satuan. Dua interval yang berbatasan pada angka yang sama tidak berarti terjadi tumpang tindih. P1 dapat selesai pada waktu lima dan P2 mulai pada waktu lima. Penjelasan ini membantu membaca batas batang Gantt tanpa menghitung titik akhir dua kali.

Tabel metrik diurutkan menurut nama proses untuk memudahkan pencarian. Urutan tabel tidak selalu sama dengan urutan eksekusi. Urutan eksekusi yang menjadi dasar perhitungan berada pada timeline. Pengguna sebaiknya membaca grafik terlebih dahulu sebelum membandingkan angka pada tabel.

3.3 Kontrak masukan dan validasi

Input yang tidak sesuai tidak boleh diam-diam berubah menjadi nilai lain. Misalnya, burst nol bukan pekerjaan selama satu satuan, dan angka 1,5 bukan otomatis satu. Versi akhir menolak input semacam itu dengan pesan yang menyebut kolom dan batas yang diperbolehkan. Validasi dilakukan di browser untuk kenyamanan dan di server karena endpoint tetap dapat dipanggil tanpa formulir.

Masukan	Batas inti	Penanganan
Jumlah proses	1 sampai 12	Tolak kosong/berlebihan
Nama proses	1 sampai 12 karakter	Tolak duplikat dan IDLE
Waktu datang	0 sampai 100	Integer
Lama proses	1 sampai 50	Integer positif
Quantum	1 sampai 20	Integer positif
Kapasitas buffer	1 sampai 10	Integer positif
Jumlah barang	1 sampai 12	Integer positif
Jeda pekerja	0 sampai 100 ms	Integer pada API

Formulir utama menyediakan tiga pilihan jeda yang diminta pekerja: sepuluh, empat puluh, dan delapan puluh milidetik. Nilai ini tidak menjamin jarak waktu antarbarang sama persis dengan parameter. Pilihan bertuliskan cepat, sedang, dan lambat membantu pengguna menghubungkan nilai jeda dengan perilaku. API mempertahankan rentang yang lebih luas untuk pengujian. Jumlah produsen dan konsumen pada halaman utama tetap satu; parameter tambahan backend dibatasi antara satu dan lima.

Server memeriksa bentuk data sebelum membaca setiap field. Daftar proses harus benar-benar berupa list yang tidak kosong, dan setiap elemennya harus berupa objek. Boolean tidak diterima sebagai angka walaupun bahasa Python memperlakukan bool sebagai turunan int. Pemeriksaan ini mencegah masukan seperti true lolos sebagai waktu satu pada API.

Pada sisi browser, kolom angka dibaca sebagai integer utuh. Masukan kosong atau pecahan menampilkan kesalahan. Tombol dan kontrol formulir dinonaktifkan selama permintaan berjalan untuk menghindari perubahan konfigurasi yang membingungkan saat hasil sedang dihitung. Setelah selesai atau gagal, kontrol dipulihkan agar pengguna dapat memperbaiki masukan dan mengulang percobaan.

Respons kesalahan input memakai status HTTP 400. Pesan ditampilkan dekat tombol pelaksanaan, bukan hanya pada console pengembang. Kondisi server tidak terhubung juga menghasilkan arahan untuk membuka aplikasi melalui launcher. Tujuannya agar pengguna awam dapat membedakan kesalahan input dari kondisi server yang belum berjalan.

3.4 Rancangan Producer-Consumer

BoundedBuffer menyimpan daftar items, kapasitas, dua semaphore dan mutex. Fungsi put() dan get() menjadi satu-satunya jalur perubahan daftar barang. Produsen tidak menambahkan barang secara langsung ke daftar, dan konsumen tidak mengambil barang tanpa memperoleh izin. Struktur ini memusatkan aturan buffer sehingga tidak tersebar di masing-masing pekerja.

```
put(barang):
    peroleh izin empty tanpa memegang mutex
    dengan mutex:
        tambahkan barang
        naikkan jumlah produksi
        catat kejadian dan salin isi buffer
    lepaskan satu izin full

get():
    peroleh izin full tanpa memegang mutex
    dengan mutex:
        ambil barang terdepan
        naikkan jumlah konsumsi
        catat kejadian dan salin isi buffer
    lepaskan satu izin empty
```

Metode _acquire() pertama mencoba memperoleh semaphore tanpa menunggu. Jika belum tersedia, kejadian BLOKIR dicatat, kemudian thread mencoba memperoleh izin dengan timeout singkat sambil memeriksa sinyal cancel. Setelah berhasil, kejadian LANJUT dicatat. Cara ini mempertahankan perilaku menunggu pada semaphore, sekaligus memungkinkan pembatalan kooperatif ketika percobaan harus dihentikan.

Pengendali run_demo() menghitung total target produksi dan membagi kuota konsumsi. Pembagian kuota juga bekerja ketika jumlah konsumen lebih besar daripada barang: sebagian konsumen memperoleh kuota nol dan langsung selesai. Dengan jumlah barang yang diketahui, tidak ada konsumen yang sengaja menunggu barang tambahan setelah seluruh target selesai.

Seluruh thread diberi satu batas waktu bersama, bukan masing-masing batas penuh yang akan menumpuk. Bila ada pekerja gagal, wrapper mencatat pesan kesalahan dan menyetel cancel. Jeda pekerja menggunakan Event.wait(), sehingga dapat dibangunkan saat pembatalan. Tes menyuntikkan kegagalan pada produsen untuk memeriksa bahwa konsumen tidak tertinggal menunggu tanpa akhir.

Respons completed hanya bernilai benar jika tidak ada error, tidak ada thread yang masih aktif, dan produksi serta konsumsi sama dengan target. Antarmuka tidak menampilkan pesan berhasil hanya karena HTTP mengembalikan objek JSON. Pemisahan status transport dan status pekerjaan ini mencegah hasil parsial disalahartikan sebagai percobaan sukses.

3.5 Pencatatan kejadian dan konsistensi data

Visualisasi hanya berguna jika rekamannya menggambarkan keadaan secara konsisten. Pada versi awal, perubahan buffer dan pencatatan kejadian dilakukan dalam bagian terpisah. Di antara kedua bagian tersebut, thread lain dapat mengubah buffer atau menulis kejadian lebih dahulu. Walaupun operasi buffer terlindungi, urutan log berisiko tidak sesuai dengan urutan perubahan yang ingin dijelaskan.

Perbaikan menempatkan perubahan barang, pembaruan jumlah, dan pembuatan snapshot di dalam mutex yang sama. Recorder tetap memiliki lock internal untuk melindungi daftar kejadian. Snapshot disalin menjadi list baru, sehingga perubahan buffer berikutnya tidak

mengubah isi kejadian yang sudah direkam. Setiap kejadian memuat pelaku, tindakan, detail, ukuran, kapasitas, daftar barang dan timestamp.

Field kejadian	Kegunaan dalam pemeriksaan
actor dan action	Menjelaskan pelaku serta tindakan
detail	Menunjukkan label barang atau alasan menunggu
buffer_size	Memeriksa batas kapasitas
items	Merekonstruksi isi buffer dan urutan FIFO
t	Menunjukkan waktu pencatatan sejak awal run

Pengujian test_core.py tidak berhenti pada pemeriksaan ukuran paling besar. Tes merekonstruksi antrean dari seluruh aksi PRODUKSI dan KONSUMSI. Pada produksi, label baru ditambahkan; pada konsumsi, label harus sama dengan barang terdepan. Setelah setiap kejadian, antrean hasil rekonstruksi dibandingkan dengan snapshot items yang direkam. Dengan demikian, kesalahan urutan log dapat terdeteksi walaupun jumlah akhir barang tetap benar.

Kejadian BLOKIR mempunyai batas penafsiran khusus. Upaya memperoleh semaphore terjadi sebelum snapshot menunggu dicatat. Thread lain dapat mengubah keadaan di antara keduanya. Karena itu, pesan mengatakan izin belum tersedia saat dicoba, bukan menjamin snapshot masih menunjukkan buffer tepat penuh atau kosong. Keterangan ini juga ditampilkan dalam bagian penjelasan tambahan antarmuka.

Pemutaran browser memakai urutan array kejadian dan snapshot asli. Browser tidak menebak isi buffer dari panjang bar animasi, tidak menyusun ulang kejadian berdasarkan timestamp yang dibulatkan, dan tidak mengubah jumlah barang saat pengguna menekan Jeda. Dengan mempertahankan data sumber, tampilan dapat dipakai untuk menjelaskan hasil yang sama dengan berkas JSON ekspor.

BAB IV IMPLEMENTASI DAN PENGUJIAN

4.1 Lingkungan dan prosedur pelaksanaan

Aplikasi dijalankan dari folder proyek dengan mulai.bat pada Windows 64-bit. Launcher memakai Python lokal bila tersedia, kemudian menjalankan run.py. Terminal dibiarkan terbuka selama aplikasi dipakai. Browser mengakses http://127.0.0.1:8000. Jika port sudah digunakan aplikasi lain, mulai.bat --port 9000 menjalankan server pada port yang berbeda.

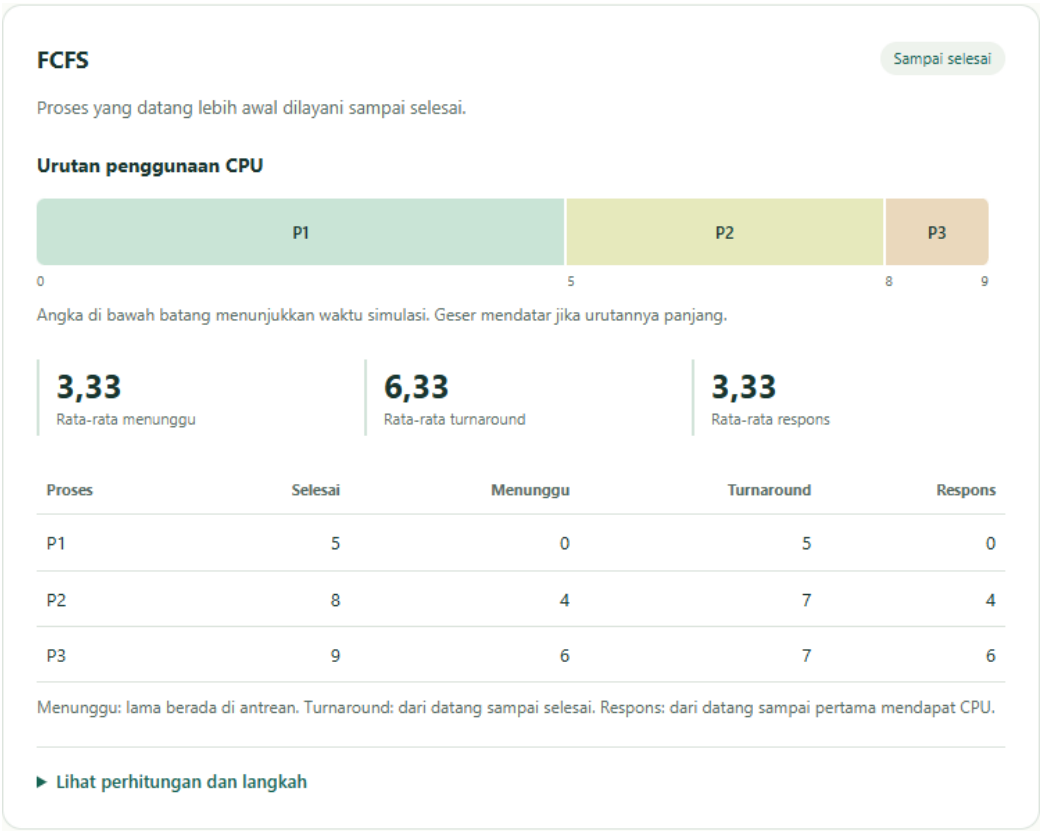
Kegiatan	Perintah
Membuka aplikasi	mulai.bat
Menggunakan port lain	mulai.bat --port 9000
Menjalankan pengujian	mulai.bat test
Menghasilkan percobaan	python scripts/collect_evidence.py

Runtime Python 3.13.1 Windows 64-bit disertakan dalam paket. Kode aplikasi menggunakan pustaka standar, sedangkan ReportLab dan PyMuPDF dipakai hanya untuk pembuatan laporan serta Playwright untuk pengujian browser. Ketergantungan tooling tidak diperlukan ketika aplikasi dijalankan. Pengujian inti memakai lingkungan Windows dan Chrome; panduan menyertakan cara menggunakan Python yang telah terpasang apabila runtime portable tidak digunakan.

Data percobaan diperoleh dari fungsi yang sama dengan endpoint aplikasi. Dataset untuk laporan ditetapkan pada experiments.json, sedangkan screenshot berasal dari eksekusi browser tersendiri. Penetapan ini menjaga tabel hasil dapat ditelusuri ke data asal tanpa menyamakan semua percobaan thread dengan satu urutan yang harus selalu identik.

4.2 Antarmuka penjadwalan CPU

Halaman scheduling memisahkan pengaturan di sebelah kiri dan hasil di sebelah kanan pada layar desktop. Pengguna memilih FCFS, Round Robin, atau perbandingan keduanya. Quantum hanya ditampilkan ketika relevan. Kolom prioritas tidak ditampilkan pada halaman utama karena tidak dipakai oleh dua algoritma inti. Tiga proses contoh sudah tersedia, tetapi perhitungan baru berjalan setelah pengguna menekan tombol.



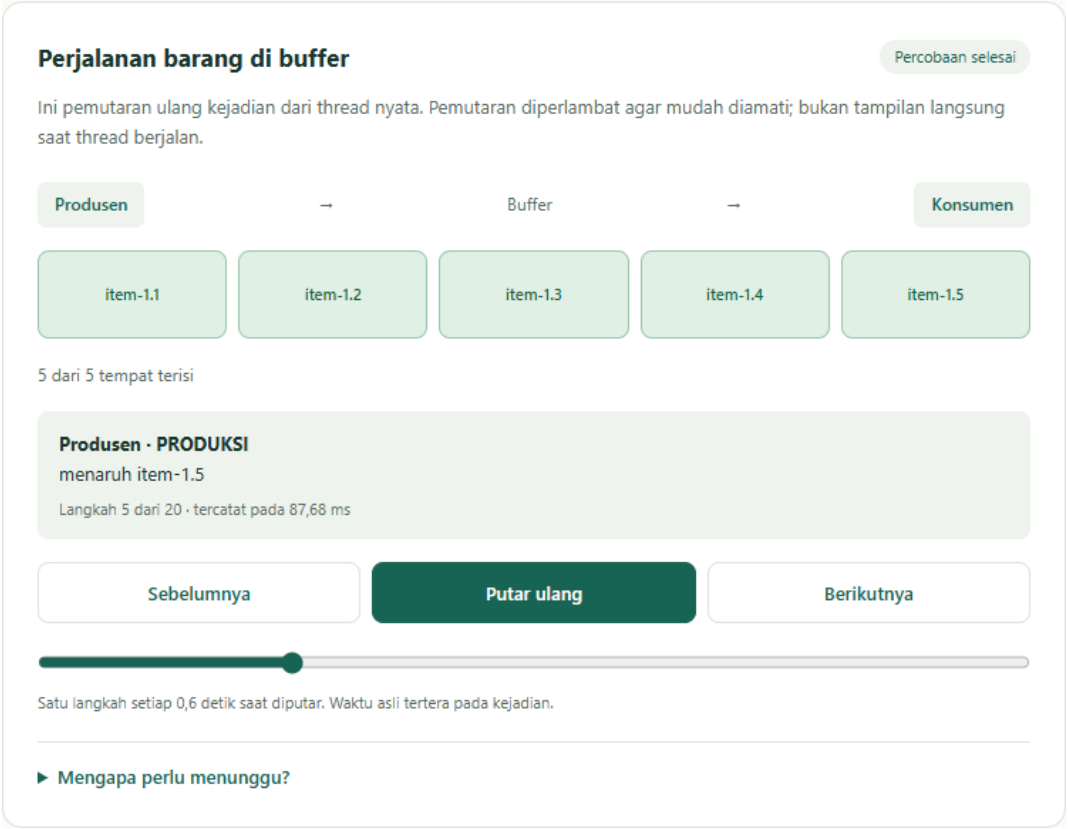
Gambar 4.1 Hasil FCFS pada contoh tiga proses yang dijalankan di browser

Hasil menempatkan Gantt chart sebelum angka, sehingga pengguna melihat urutan pekerjaan terlebih dahulu. Rata-rata menunggu, turnaround, dan respons kemudian ditampilkan sebagai tiga ukuran utama. Tabel di bawahnya memungkinkan pengguna memeriksa angka setiap proses. Detail perhitungan dan log keputusan disimpan di bagian yang dapat dibuka tanpa memenuhi layar awal dengan seluruh penjelasan.

Saat grafik panjang, area grafik dapat digeser mendatar tanpa menyebabkan seluruh halaman melebar. Warna proses konsisten di dalam hasil dan label nama tetap tersedia. Penjelasan angka disajikan dalam bahasa Indonesia. Tombol simpan hasil JSON menyediakan data yang dapat digunakan untuk dokumentasi atau pemeriksaan ulang. Screenshot ini berasal dari aplikasi yang berjalan, bukan rancangan tampilan sebelum implementasi.

4.3 Antarmuka Producer-Consumer

Halaman Producer–Consumer menampilkan kontrol kapasitas, jumlah barang, dan kecepatan dua pekerja. Tombol preset produsen lebih cepat dan konsumen lebih cepat memudahkan penggantian skenario. Setelah percobaan selesai, hasil menampilkan tempat penyimpanan berbentuk kotak dan tombol Sebelumnya, Putar ulang, serta Berikutnya. Slider dapat dipakai untuk langsung memilih posisi rekaman.



Gambar 4.2 Pemutaran rekaman buffer dari percobaan thread nyata

Kotak berwarna memuat label barang, sementara tempat kosong diberi teks kosong. Bagian kejadian menjelaskan siapa melakukan apa, nomor langkah, dan timestamp asli. Pemutaran otomatis berpindah setiap 0,6 detik agar kejadian dapat diikuti. Antarmuka menyatakan bahwa ini pemutaran ulang setelah percobaan selesai sehingga pengguna tidak menyamakan animasi dengan streaming eksekusi thread.

Ringkasan hasil akhir ditempatkan terpisah dari keadaan buffer pada langkah terpilih. Dengan demikian, angka delapan barang selesai tidak dimaksudkan sebagai jumlah barang di dalam buffer saat slider berada di tengah rekaman. Informasi menunggu dan jumlah thread belum selesai membantu menilai akhir percobaan. Pada ponsel, pengaturan dan hasil ditumpuk secara vertikal, sedangkan kotak buffer tetap berada dalam lebar layar.

4.4 Strategi dan cakupan pengujian

Pengujian dilakukan pada beberapa lapisan karena keberhasilan satu lapisan tidak membuktikan keberhasilan lapisan lain. Fungsi dapat menghasilkan angka benar tetapi frontend gagal menampilkan respons. Sebaliknya, tampilan dapat terlihat rapi walaupun rumusnya salah. Oleh karena itu, pengujian manual, fungsi, HTTP dan browser digunakan untuk menjawab pertanyaan yang berbeda.

Lapisan	Jumlah yang dicatat	Hasil
HTTP dan modul tambahan	59 pemeriksaan pada test_api.py	Lulus

Lapisan	Jumlah yang dicatat	Hasil
Scheduling lama	20 pemeriksaan pada test_scheduling.py	Lulus
Regresi inti	21 kasus unittest pada test_core.py	Lulus
Browser	20 alur pemeriksaan	Lulus
Percobaan buffer laporan	15 run pada tiga skenario	Selesai dan invariant terpenuhi

Istilah pemeriksaan dan kasus unittest tidak disamakan dengan jumlah assertion. Satu kasus unittest dapat memiliki beberapa assertion atau loop subkasus. Penghitungan tersebut dilaporkan terpisah untuk menghindari klaim bahwa seluruh angka merupakan unit pengukuran yang sama. Log asli tersedia pada docs/evidence dan dapat dibaca tanpa mengandalkan ringkasan laporan.

Tes scheduling membandingkan timeline dan metrik terhadap hasil manual, memeriksa kedatangan tepat pada batas quantum, quantum besar yang menyamai FCFS, kasus idle, nama ganda, nama IDLE, dan quantum tidak valid. Tes buffer memeriksa dua perbedaan kecepatan, banyak pekerja, konsumen lebih banyak daripada barang, serta kegagalan dan timeout. Jalur pembatalan diuji agar tidak hanya jalur sukses yang mendapat perhatian.

Pengujian browser dilakukan melalui Chrome headless pada viewport 1440 kali 1080 dan 390 kali 844. Alur yang diperiksa mencakup FCFS tunggal, RR tunggal, perbandingan, validasi input, penambahan dan penghapusan proses, ekspor JSON, pemutaran rekaman, preset kecepatan, materi tambahan, serta overflow halaman. Tidak ada pageerror JavaScript yang tercatat selama alur tersebut.

Batas pengujian tetap ada. Tidak semua browser atau ukuran layar dicoba, dan tidak semua interleaving thread dapat dijalankan satu per satu. Hasil lulus mendukung kesiapan demo pada lingkungan yang diuji. Evaluasi pedagogis terhadap peserta belum dilakukan, sehingga hasil teknis ini tidak digunakan untuk mengklaim peningkatan pemahaman belajar.

4.5 Verifikasi manual FCFS

Data contoh terdiri dari tiga proses. P1 datang pada nol dengan kebutuhan CPU lima, P2 datang pada satu dengan kebutuhan tiga, dan P3 datang pada dua dengan kebutuhan satu. Semua proses menggunakan nama unik. FCFS mempertahankan P1 sampai selesai sehingga kedatangan P2 dan P3 tidak memotong pekerjaan P1.

Proses	Datang	Lama CPU
P1	0	5
P2	1	3
P3	2	1

P1 mulai pada nol dan selesai pada lima. Pada waktu lima, P2 sudah menunggu sejak satu dan P3 sejak dua. P2 datang lebih awal sehingga mendapat giliran kedua, berjalan sampai delapan. P3 lalu memakai CPU selama satu satuan dan selesai pada sembilan. Tidak ada interval idle karena selalu ada pekerjaan yang tersedia setelah P1 mulai.

```
P1 0-5 | P2 5-8 | P3 8-9
```

Proses	Selesai	Tunggu	Turnaround	Respons
P1	5	0	5	0
P2	8	4	7	4
P3	9	6	7	6

Untuk P1, turnaround lima dikurangi nol menghasilkan lima, kemudian dikurangi burst lima menghasilkan waktu tunggu nol. Untuk P2, turnaround delapan dikurangi satu menghasilkan tujuh; waktu tunggu tujuh dikurangi tiga adalah empat. Untuk P3, turnaround sembilan dikurangi dua menghasilkan tujuh; waktu tunggu tujuh dikurangi satu adalah enam.

Jumlah waktu tunggu adalah sepuluh, sehingga rata-ratanya sepuluh dibagi tiga atau 3,33 setelah pembulatan. Jumlah turnaround adalah sembilan belas sehingga rata-ratanya 6,33. Karena setiap proses hanya memiliki satu giliran aktif, respons sama dengan waktu tunggu pada contoh ini. Pergantian terjadi dari P1 ke P2, lalu P2 ke P3, sehingga jumlahnya dua.

Nilai manual ini dijadikan expected result pada test_core.py. Pemeriksaan membandingkan seluruh urutan segmen dan metrik tiap proses, bukan hanya memeriksa bahwa respons API memiliki field. Dengan demikian, perubahan implementasi yang tetap menghasilkan JSON tetapi mengubah urutan atau angka akan terdeteksi pada contoh ini.

4.6 Verifikasi manual Round Robin

Contoh yang sama dijalankan dengan quantum dua. P1 memperoleh giliran pertama dari nol sampai dua dan masih mempunyai tiga satuan pekerjaan. Pada waktu dua, P2 dan P3 sudah datang. Keduanya masuk antrean sebelum P1 dikembalikan ke belakang. Antrean berikutnya adalah P2, P3, P1.

Giliran	Proses	Interval	Sisa setelah giliran
1	P1	0–2	3
2	P2	2–4	1
3	P3	4–5	0
4	P1	5–7	1

Giliran	Proses	Interval	Sisa setelah giliran
5	P2	7–8	0
6	P1	8–9	0

P2 memakai dua satuan pada giliran kedua dan tersisa satu. P3 hanya memerlukan satu satuan sehingga selesai pada waktu lima. P1 kemudian menggunakan dua satuan lagi, diikuti P2 yang menyelesaikan satu satuan terakhir. Terakhir, P1 menyelesaikan sisa pekerjaannya pada waktu sembilan.

Waktu akhir	Proses baru selesai giliran	Antrean berikutnya
2	P1, sisa 3	P2, P3, P1
4	P2, sisa 1	P3, P1, P2
5	P3 selesai	P1, P2
7	P1, sisa 1	P2, P1
8	P2 selesai	P1
9	P1 selesai	Kosong

Proses	Selesai	Tunggu	Turnaround	Respons
P1	9	4	9	0
P2	8	4	7	1
P3	5	2	3	2

P1 selesai pada sembilan, sehingga turnaround sembilan dan waktu tunggu empat. P2 tetap selesai pada delapan dengan turnaround tujuh dan waktu tunggu empat. P3 selesai lebih cepat, pada lima, dengan turnaround tiga dan waktu tunggu dua. Total waktu tunggu masih sepuluh, sama dengan FCFS pada data ini, walaupun pembagiannya berubah.

Waktu pertama berjalan adalah nol untuk P1, dua untuk P2, dan empat untuk P3. Setelah dikurangi waktu datang masing-masing, responsnya nol, satu, dan dua. Rata-ratanya satu, lebih kecil daripada respons FCFS 3,33 pada contoh yang sama. Ada enam segmen aktif dengan pergantian identitas pada setiap batas, sehingga lima pergantian antarproses dihitung.

Kasus ini juga memverifikasi kedatangan tepat pada batas quantum: P3 datang pada waktu dua. Tes mengharuskan urutan tersebut tetap sesuai aturan yang didokumentasikan. Perbandingan ini memberi alasan konkret mengapa respons dapat membaik tanpa perubahan rata-rata waktu tunggu pada dataset tertentu.

4.7 Pengaruh quantum

Percobaan mengubah quantum dengan data P1, P2 dan P3 yang sama. Nilai quantum yang digunakan adalah satu, dua, tiga, lima, sepuluh dan dua puluh. Semua hasil berikut berasal dari scripts/collect_evidence.py dan tersimpan pada docs/evidence/experiments.json. Setiap perubahan parameter dilakukan pada fungsi yang sama dengan fungsi yang dipanggil endpoint aplikasi.

Quantum	Tunggu	Turnaround	Respons	Pergantian
1	2,67	5,67	0,33	7
2	3,33	6,33	1,0	5
3	3,33	6,33	2,0	3
5	3,33	6,33	3,33	2
10	3,33	6,33	3,33	2
20	3,33	6,33	3,33	2

Pada quantum satu, rata-rata respons adalah 0,33 dan terdapat tujuh pergantian antarproses. Quantum dua menghasilkan respons satu dengan lima pergantian. Quantum tiga memberi respons dua dengan tiga pergantian. Untuk quantum lima atau lebih besar, setiap burst contoh dapat selesai pada giliran pertama sehingga urutan dan metriknya sama dengan FCFS.

Rata-rata waktu tunggu pada quantum satu adalah 2,67, sedangkan pada quantum dua dan tiga nilainya 3,33. Angka tersebut merupakan hasil dari susunan kedatangan dan panjang pekerjaan tertentu. Nilai ini tidak membuktikan bahwa quantum satu akan selalu meminimalkan waktu tunggu pada dataset lain. Pengguna dapat mengganti input untuk menguji apakah pola yang sama masih terjadi.

Total waktu simulasi tetap sembilan untuk seluruh percobaan karena jumlah burst sembilan, tidak ada idle, dan model tidak menambahkan overhead. Karena itu, tabel tidak dapat digunakan untuk menyimpulkan quantum kecil tidak memiliki biaya di sistem nyata. Peningkatan jumlah pergantian menunjukkan perubahan frekuensi peralihan, sedangkan durasi biaya peralihan berada di luar cakupan model.

Perbandingan harus menyatakan ukuran dan dataset secara bersamaan. Pada tiga proses contoh, quantum dua memperbaiki rata-rata respons dibanding FCFS, sementara rata-rata waktu tunggu sama. Hasil ini menunjukkan bahwa perubahan respons tidak selalu diikuti perubahan rata-rata tunggu. Kesimpulan tetap dibatasi pada parameter percobaan yang dicatat.

4.8 Pekerjaan panjang dan waktu idle

Dua kasus tambahan membantu memeriksa batas perilaku model. Kasus pertama menempatkan pekerjaan panjang di depan dua pekerjaan pendek yang datang bersamaan. P1 memerlukan dua puluh empat satuan, sedangkan P2 dan P3 masing-masing memerlukan tiga. Karena waktu datang sama, nama proses menentukan urutan awal.

Algoritma	Rata-rata tunggu	Rata-rata respons	Pergantian
fcfs	17,0	17,0	2
rr2	6,33	2,0	6

Pada FCFS, P2 menunggu dua puluh empat dan P3 menunggu dua puluh tujuh satuan. Rata-rata waktu tunggu menjadi tujuh belas. Angka tersebut dapat diperoleh tanpa membaca kode: cukup jumlahkan lama pekerjaan yang berada di depan masing-masing proses. Pada RR quantum dua, kedua proses pendek dapat memperoleh CPU sebelum pekerjaan panjang selesai. Hasil tabel menunjukkan perubahan pengalaman menunggu dan memperoleh giliran pertama pada konfigurasi tersebut.

Kasus kedua hanya memiliki P1 yang baru datang pada waktu lima dan membutuhkan tiga satuan CPU. Program mencatat IDLE dari nol sampai lima, kemudian P1 dari lima sampai delapan. Waktu tunggu dan respons P1 nol karena tidak ada proses lain yang mendahuluinya ketika sudah datang. Turnaround P1 adalah tiga, bukan delapan.

Ukuran kasus idle	Nilai
Waktu total	8
Waktu CPU aktif	3
Utilisasi model	37,5%
Rata-rata tunggu	0
Pergantian antarproses	0

Perbedaan antara total waktu dan turnaround penting untuk mencegah salah tafsir. Total waktu diukur sejak awal sumbu simulasi nol, sedangkan turnaround diukur sejak kedatangan suatu proses. Utilisasi model pada contoh ini adalah tiga dibagi delapan, atau 37,5 persen. Ini sekadar rasio waktu aktif terhadap rentang model, bukan pengukuran Task Manager.

Pengujian juga mencakup jeda di antara dua proses. Peralihan dari proses ke IDLE lalu ke proses berikutnya tidak masuk hitungan pergantian langsung antarproses sesuai definisi laporan. Dengan mencatat aturan tersebut, hasil idle dan hasil tanpa idle tetap dapat dibandingkan secara konsisten.

4.9 Percobaan produsen lebih cepat

Konfigurasi percobaan ini menggunakan kapasitas lima, delapan barang, satu produsen dan satu konsumen. Jeda produsen sepuluh milidetik dan jeda konsumen delapan puluh milidetik. Setiap skenario dijalankan lima kali. Data pada tabel adalah hasil run_demo() yang benar-benar dijalankan, bukan contoh angka yang diketik untuk menggambarkan perilaku.

Ulangan	Dibuat	Diambil	Isi puncak	Tunggu prod.	Tunggu kons.
1	8	8	5	2	0
2	8	8	5	2	0
3	8	8	5	2	0
4	8	8	5	2	0
5	8	8	5	2	0

Produsen mampu mencoba memasukkan barang lebih sering daripada konsumen mengambilnya. Isi buffer kemudian bertambah sampai kapasitasnya tercapai. Setelah tidak memperoleh izin empty, produsen menunggu. Konsumen yang mengambil barang melepaskan izin empty sehingga produsen dapat melanjutkan. Pada semua ulangan yang dicatat, isi tertinggi mencapai lima dan tidak melampaui kapasitas.

Semua ulangan berakhir dengan delapan barang diproduksi dan delapan barang dikonsumsi. Sisa buffer nol dan daftar thread yang belum selesai kosong. Hasil tersebut menunjukkan bahwa jeda menunggu tidak menghilangkan barang. Produsen tetap menyelesaikan targetnya setelah tersedia kesempatan untuk menaruh barang berikutnya.

Jumlah kejadian menunggu adalah hitungan upaya memperoleh izin yang sempat gagal, bukan durasi total menunggu. Satu kejadian BLOKIR dapat diikuti interval waktu yang panjang atau pendek. Karena itu, angka dua kejadian menunggu tidak boleh dibaca sebagai dua milidetik. Waktu pada log memakai timestamp sejak awal percobaan dan dapat digunakan untuk menelusuri urutan, dengan tetap memperhatikan bahwa pencatatan sendiri menambah pekerjaan.

Hasil screenshot dari browser dapat memiliki jumlah menunggu yang berbeda dari tabel karena diambil pada pemanggilan percobaan lain. Variasi itu bukan alasan mengganti angka laporan agar terlihat seragam. Tabel berasal dari data ekspor yang ditetapkan sebagai bukti percobaan, sedangkan screenshot mendokumentasikan bentuk antarmuka dan contoh keadaan yang pernah ditampilkan.

Demo ini cocok untuk menjelaskan fungsi pembatasan kapasitas. Tanpa izin slot, menambah barang terus-menerus akan bertentangan dengan model buffer terbatas. Sistem memilih menunda produsen sampai terdapat ruang, sehingga kapasitas tetap dipatuhi selama proses produksi dan konsumsi.

4.10 Percobaan konsumen lebih cepat

Pada skenario kedua, jeda produsen diubah menjadi delapan puluh milidetik dan jeda konsumen menjadi sepuluh milidetik. Parameter lain tetap sama. Perubahan satu pasangan jeda ini menjaga perbandingan tetap mudah dipahami: jumlah barang dan kapasitas tidak berubah, tetapi pekerja yang lebih sering siap melakukan tindakan berganti.

Ulangan	Dibuat	Diambil	Isi puncak	Tunggu prod.	Tunggu kons.
1	8	8	1	0	8
2	8	8	1	0	8
3	8	8	1	0	8
4	8	8	1	0	8
5	8	8	1	0	8

Konsumen mencoba mengambil barang sebelum produsen sempat menyediakan barang berikutnya. Karena full belum memberi izin, konsumen menunggu. Setelah produsen memasukkan satu barang, izin full dilepas dan konsumen dapat melanjutkan. Pada ulangan yang tercatat, puncak isi buffer satu karena barang segera diambil sebelum sempat terjadi penumpukan yang lebih besar.

Walaupun konsumen lebih cepat, jumlah konsumsi tidak boleh melampaui jumlah barang yang diproduksi. Konsumen tidak membuat barang baru dan tidak mengambil nilai dari daftar kosong. Sifat tersebut diperiksa melalui urutan label barang: label yang diambil harus sama dengan label pertama dalam antrean hasil produksi pada saat itu.

Semua percobaan selesai dengan target delapan barang. Hasil ini menunjukkan perbedaan antara kemampuan mencoba lebih sering dan keberadaan barang yang dapat diambil. Kecepatan konsumen saja tidak cukup untuk meningkatkan jumlah hasil akhir ketika produsen masih menyediakan barang secara bertahap. Pengguna dapat melihat alasan menunggu langsung pada pesan kejadian di pemutaran ulang.

Kondisi konsumen menunggu tidak diberi warna kegagalan pada aplikasi karena merupakan bagian normal dari mekanisme sinkronisasi. Pesan gagal hanya digunakan untuk input tidak valid, server yang tidak terhubung, atau percobaan yang tidak selesai. Pemilihan bahasa antarmuka ini membantu menghindari pemahaman bahwa semua bentuk blocking merupakan kesalahan.

Perbandingan kedua skenario memperlihatkan pembagian tanggung jawab semaphore: empty berkaitan dengan kesempatan produsen menaruh barang, sedangkan full berkaitan dengan kesempatan konsumen mengambil barang. Mutex tetap dibutuhkan pada kedua skenario karena melindungi perubahan daftar dan snapshot terlepas dari pihak mana yang sedang lebih cepat.

4.11 Jeda sama dan variasi thread

Skenario ketiga memberikan jeda empat puluh milidetik kepada kedua pekerja. Secara parameter, produsen dan konsumen mempunyai jeda yang sama. Namun, kesamaan jeda tidak memaksa keduanya dieksekusi dalam urutan yang persis sama pada setiap percobaan. Awal thread, waktu bangun, perolehan lock, dan penjadwalan oleh lingkungan eksekusi masih dapat berbeda.

Ulangan	Dibuat	Diambil	Isi puncak	Tunggu prod.	Tunggu kons.
1	8	8	1	0	4
2	8	8	1	0	6
3	8	8	1	0	2
4	8	8	1	0	4
5	8	8	1	0	3

Tabel menunjukkan hasil akhir jumlah barang tetap konsisten, sedangkan kejadian menunggu konsumen dapat berbeda. Bukti ini membantu menjelaskan mengapa satu percobaan thread tidak cukup untuk menyimpulkan semua urutan kejadian. Pengujian harus menekankan sifat yang wajib selalu benar, seperti kapasitas, identitas barang, urutan FIFO, dan penghentian pekerja, bukan mengunci seluruh timestamp menjadi satu expected result.

Untuk scheduling, data dan aturan yang sama menghasilkan urutan deterministik dalam fungsi simulasi. Untuk Producer–Consumer, program menjalankan thread nyata sehingga urutan pencatatan bergantung pada interleaving. Perbedaan sifat tersebut sengaja dipertahankan dan dijelaskan. Mengganti thread nyata dengan animasi yang urutannya dibuat tetap akan mengubah apa yang diperagakan oleh modul kedua.

Seluruh lima belas percobaan buffer yang dicatat menghasilkan produksi dan konsumsi sesuai target, buffer akhir kosong, serta tidak menyisakan thread aktif. Pernyataan ini berlaku untuk lima belas run yang disimpan pada berkas bukti. Pernyataan tersebut tidak sama dengan bukti formal bahwa setiap lingkungan dan setiap parameter akan selalu selesai; kasus uji regresi dan mekanisme pembatalan melengkapi bukti empiris ini.

Jika percobaan diulang pada laptop lain, jumlah menunggu dan timestamp boleh berubah. Untuk membandingkan hasil, simpan konfigurasi bersama JSON hasil. Fokuskan pembahasan pada apakah invariant tetap terpenuhi dan apakah perubahan kecepatan memperlihatkan keadaan yang dapat dijelaskan. Jangan mengganti hasil aktual hanya agar cocok dengan satu tangkapan layar atau contoh sebelumnya.

Analisis ini juga membedakan waktu model dengan waktu eksekusi nyata. Angka pada scheduling merupakan satuan abstrak, sementara timestamp buffer berasal dari pengukuran perf_counter() dalam milidetik. Kedua jenis angka tidak digabungkan menjadi satu ukuran performa karena berasal dari proses pengukuran yang berbeda.

4.12 Jeda masukan dan interval kejadian aktual

Konfigurasi sepuluh milidetik adalah parameter jeda yang diminta oleh pekerja. Program memanggil Event.wait(jeda) sebelum mencoba put atau get. API ini menunggu perubahan flag atau berakhirnya timeout [4]. Sesudahnya masih ada pekerjaan penjadwalan thread, perolehan semaphore/mutex, perubahan buffer dan pencatatan event. Oleh sebab itu, parameter jeda, durasi satu pemanggilan wait, dan selisih waktu antar-event adalah tiga besaran yang berbeda.

Data lama tidak merekam awal dan akhir setiap pemanggilan wait. Analisis tambahan menggunakan empat selisih timestamp di antara lima event PRODUKSI pertama pada setiap run produsen cepat. Timestamp berasal dari perf_counter() yang dibulatkan ke dua desimal milidetik. Ini mengukur interval antarpencatatan produksi, bukan waktu tidur murni. Metode dan hash data asal tersimpan pada timing-analysis.json; tidak ada run asli yang diganti.

Run	Jeda diminta	Interval minimum	Interval maksimum	Rata-rata interval
1	10 ms	15,29 ms	21,82 ms	17,72 ms
2	10 ms	15,02 ms	18,53 ms	16,84 ms
3	10 ms	15,42 ms	17,55 ms	16,41 ms
4	10 ms	14,54 ms	17,22 ms	15,83 ms
5	10 ms	15,84 ms	16,6 ms	16,28 ms

Dua puluh interval awal yang dianalisis berada dalam rentang 14,54 sampai 21,82 ms. Nilainya lebih besar daripada parameter sepuluh milidetik pada dataset ini. Bukti tersebut cukup untuk menolak anggapan bahwa pilihan 10 ms menjamin satu barang tercatat setiap 10 ms. Bukti ini tidak mengisolasi kontribusi resolusi timer Windows, penjadwalan CPU, kontensi lock, atau pekerjaan pencatatan. Penetapan satu penyebab tunggal memerlukan instrumentasi tambahan yang belum dilakukan.

Penghitungan BLOKIR dari event juga diulang secara terpisah dari field ringkasan. Kelima run produsen cepat masing-masing berisi dua kejadian produsen gagal memperoleh izin pada percobaan awal, sesuai tabel hasil. Perhitungan ideal yang menganggap kedua pekerja berjalan tepat setiap 10 dan 80 ms dapat menghasilkan jumlah berbeda. Perhitungan ideal tersebut menjelaskan model waktu yang lain dan tidak menggantikan pengamatan eksekusi thread.

Reproduksi pada Linux, macOS, versi Python lain, atau beban komputer yang berbeda boleh menghasilkan timestamp dan jumlah penantian berbeda. Yang diperiksa lintas run ialah identitas serta jumlah barang, FIFO, kapasitas buffer, dan status penyelesaian. Pengujian pada lingkungan selain Windows belum dilakukan. Konfigurasi dan hasil JSON perlu disimpan bersama apabila dilakukan perbandingan baru.

Pemutaran browser menggunakan langkah tetap 0,6 detik agar urutan dapat diamati. Kecepatan ini juga tidak sama dengan jeda asli pekerja. Menekan Jeda hanya menghentikan

pemutaran log; kedua pekerja sudah selesai sebelum hasil diterima browser. Dengan membedakan ketiga lapisan waktu tersebut, perubahan visual tidak disalahartikan sebagai perubahan kecepatan produksi pada run yang sudah berlangsung.

4.13 Pengujian dan perbaikan

Pengujian dan perbaikan difokuskan pada perilaku yang dapat diperiksa kembali. Tabel berikut menghubungkan masukan pemicu, respons yang diharapkan, dan bukti regresinya. Penilaian tidak hanya berdasarkan keberhasilan server mengembalikan JSON, tetapi juga angka hasil, penanganan kegagalan, dan kesinambungan pelayanan setelah masukan ditolak.

Pemicu	Perilaku yang diperiksa	Bukti
Satu proses, burst 5, quantum 2	Tiga giliran; nol pergantian antarproses	test_rr_single_process_not_switch
Kedatangan di akhir quantum	Proses baru mendahului proses yang diantrekan ulang	test_rr_manual_boundary_arrival
IDLE atau burst di luar rentang	Masukan ditolak, tanpa penggantian nilai diam-diam	Tes validasi dan browser
Error pekerja atau timeout	Hasil gagal dan pekerja dapat dibatalkan	Tes error/timeout
JSON 10.000 tingkat atau angka 5.000 digit	HTTP 400; request valid berikutnya tetap berhasil	test_complex_json_rejected_and_server_recovers
Perubahan buffer oleh banyak pekerja	Setiap snapshot cocok dengan rekonstruksi FIFO	test_multiple_workers_trace

Kasus JSON menunjukkan bahwa ukuran permintaan saja tidak cukup untuk menyatakan sebuah input dapat diproses. Badan permintaan yang masih di bawah batas ukuran dapat memiliki struktur yang terlalu dalam atau angka yang melampaui batas konversi interpreter. Handler menangani RecursionError dan ValueError dari decoding sebagai masukan yang ditolak. Regresi mengirim request normal sesudahnya untuk memastikan layanan tetap dapat digunakan.

Pada pengujian awal kasus baru, integer panjang menyebabkan koneksi ditutup tanpa respons JSON. Setelah penanganan diperbaiki, kedua subkasus ditolak dengan status 400 dan request normal menghasilkan status 200. Log kegagalan serta hasil setelah perbaikan disimpan terpisah. Kedalaman subkasus ditetapkan 10.000 agar benar-benar memicu batas decoder pada lingkungan yang digunakan.

BAB V PENUTUP

5.1 Kesimpulan

Aplikasi memperagakan dua materi melalui FCFS/Round Robin dan Producer-Consumer. Perhitungan manual sesuai dengan timeline serta metrik program. Pada contoh tiga proses, RR quantum dua menurunkan rata-rata respons dari 3,33 menjadi satu, sementara rata-rata tunggu tetap 3,33. Kesimpulan ini berlaku pada dataset tersebut dan bukan pemeringkatan universal algoritma.

Lima belas run buffer menyelesaikan delapan barang per run, menjaga kapasitas dan FIFO, serta tidak menyisakan thread aktif. Parameter jeda tidak menjamin interval kejadian yang sama; data masukan 10 ms memperlihatkan interval produksi awal 14,54-21,82 ms. Rekaman yang konsisten memungkinkan urutan tersebut ditelusuri kembali tanpa mengganti angka pengamatan.

Pengujian fungsi, HTTP, browser dan kasus kegagalan mendukung kesesuaian implementasi pada lingkungan yang diperiksa. Penyajian grafik dan snapshot menghubungkan aturan dengan kejadian konkret. Pengungkapan batas model tetap diperlukan agar demonstrasi tidak disamakan dengan seluruh mekanisme sistem operasi atau komunikasi dua proses terpisah.

5.2 Keterbatasan

Scheduling menggunakan satu CPU konseptual, burst diketahui, tanpa I/O dan tanpa biaya waktu perpindahan. Buffer menggunakan thread dalam satu proses dan hasil diputar setelah percobaan selesai. Pengujian berlangsung pada Windows/Chrome; lingkungan lain dan seluruh interleaving belum dicakup.

Bounded waiting dan ketiadaan starvation tidak dibuktikan secara formal. Interval event tidak mengisolasi durasi wait atau penyebab variasinya. Evaluasi pemahaman peserta belum dilakukan, sehingga kelulusan teknis tidak dianggap sebagai bukti peningkatan hasil belajar.

5.3 Saran pengembangan

Pengembangan berikutnya dapat memodelkan biaya context switch dan menampilkan antrean siap secara lebih rinci. Pada buffer, instrumentasi awal/akhir wait dapat memisahkan penantian dari pekerjaan pencatatan. Setiap perluasan perlu disertai definisi pengukuran dan kasus pembanding. Evaluasi dengan peserta dapat digunakan untuk menilai apakah alat peraga membantu menjelaskan konsep secara mandiri.

DAFTAR PUSTAKA

[1] Remzi H Arpaci-Dusseau dan Andrea C Arpaci-Dusseau. *Operating Systems Three Easy Pieces Bab 4 The Abstraction The Process*. Diakses 24 Oktober 2026. https://pages.cs.wisc.edu/~remzi/OSTEP/cpu-intro.pdf

[2] Remzi H Arpaci-Dusseau dan Andrea C Arpaci-Dusseau. *Operating Systems Three Easy Pieces Bab 7 Scheduling Introduction*. Diakses 24 Oktober 2026. https://pages.cs.wisc.edu/~remzi/OSTEP/cpu-sched.pdf

[3] Remzi H Arpaci-Dusseau dan Andrea C Arpaci-Dusseau. *Operating Systems Three Easy Pieces Bab 26 Concurrency An Introduction*. Diakses 24 Oktober 2026. https://pages.cs.wisc.edu/~remzi/OSTEP/threads-intro.pdf

[4] Python Software Foundation. *Dokumentasi Python 3.13 threading*. Diakses 24 Oktober 2026. https://docs.python.org/3.13/library/threading.html

[5] Remzi H Arpaci-Dusseau dan Andrea C Arpaci-Dusseau. *Operating Systems Three Easy Pieces Bab 31 Semaphores*. Diakses 24 Oktober 2026. https://pages.cs.wisc.edu/~remzi/OSTEP/threads-sema.pdf

[6] Remzi H Arpaci-Dusseau dan Andrea C Arpaci-Dusseau. *Operating Systems Three Easy Pieces Bab 28 Locks*. Diakses 24 Oktober 2026. https://pages.cs.wisc.edu/~remzi/OSTEP/threads-locks.pdf

[7] Abraham Silberschatz, Peter B. Galvin dan Greg Gagne. *Operating System Concepts, edisi 10. Slide pendamping Bab 3: Processes, slide 8-10 dan 16-17*. Diakses 25 Oktober 2026. https://os-book.com/OS10/slide-dir/PPTX-dir/ch3.pptx

[8] Abraham Silberschatz, Peter B. Galvin dan Greg Gagne. *Operating System Concepts, edisi 10. Slide pendamping Bab 5: CPU Scheduling, slide 10 dan 21-23*. Diakses 25 Oktober 2026. https://os-book.com/OS10/slide-dir/PPTX-dir/ch5.pptx

[9] Abraham Silberschatz, Peter B. Galvin dan Greg Gagne. *Operating System Concepts, edisi 10. Slide pendamping Bab 6: Synchronization Tools, slide 8 dan 56*. Diakses 25 Oktober 2026. https://os-book.com/OS10/slide-dir/PPTX-dir/ch6.pptx

[10] Andrew S. Tanenbaum. *Modern Operating Systems, edisi 2 (2001). Sampel resmi bagian 2.2: Threads, hlm. 81-83*. Diakses 25 Oktober 2026. https://www.cs.vu.nl/~ast/books/mos2/sample-2.pdf

[11] Edsger W. Dijkstra. *Cooperating Sequential Processes, EWD123, bagian 3.2: The Synchronizing Primitives*. Diakses 25 Oktober 2026. https://www.cs.utexas.edu/~EWD/transcriptions/EWD01xx/EWD123.html